

Design and Analysis of Algorithms

Module X1: region-based memory management

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

What are we doing here?

This is a class about algorithms.

The first part of class was very theoretical, but over time, we will get more pragmatic.

Memory management is classically a somewhat slow operation, originally typically $O(n)$ or $O(\lg n)$. However, recently it has improved to $O(1)$ for small, constant time allocations with most allocators.

We're going to learn how to beat the default allocator with a constant ratio improvement.

It's not a big- O improvement, but sometimes constant factors are meaningful...

The era of the memory map

In the days before persistent operating systems, it was common for programs to explicitly statically lay out every piece of memory.

[Here is a memory map](#) for the layout of RAM in the original Pokemon games. [Documented here](#). You can see that every variable the game uses has an explicit location that is statically determined. It is the same every time the game boots.

Early cheat systems like the Gameshark would intercept certain memory addresses and replace them. They were always the same, so this would work.

The era of the memory map (2)

You might think that this is like putting everything on the stack, but it isn't quite the same. The stack grows and shrinks as needed. Static memory is always there.

It's more like making everything a global.

Despite the advice you often hear that "globals are bad", in systems like this, static memory is quite useful. You can 100% guarantee that you will never run out of memory.

Pokemon is a fairly buggy game, but under normal play conditions, there will never be a memory panic.

Accessing a static address means no need to push it or move it when calling a function, so there are potential speed benefits as well.

What's wrong with static memory?

To be honest, I feel like we might have abandoned this idea too quickly. Many modern games and other systems could benefit from explicit static memory.

The issue, though, is that it doesn't scale for most utilities. Consider old versions of Windows notepad: they had a 64 kilobyte buffer for text.

If you needed to edit 65 kilobytes, too bad.

Could they just increase it to a 64 *megabyte* buffer? That would be enough for most purposes...

But then it would be wasteful most of the time.

Stacks

So then we have the stack. Stacks are so useful, most processors have a built-in register that stores a pointer to a stack.

Why are they useful? Because there is a common pattern of memory usage: always allocating and freeing in reverse order.

That pattern happens with function calls and local variables. When you call a function, it can use the stack pointer to organize its local storage.

Stacks (2)

If that function calls another function, as long as the other function uses memory below the first function's stack pointer, it won't interfere.

When a function returns, it restores the stack to where it was. That memory can easily be overwritten.

Usually the stack grows down. The idea is that static memory would be at the top of a typical address space, then the heap, and stack would be at the end. This would allow the stack and heap to grow towards each other and maximize the space available.

Stacks (3)

Many programming languages, such as C, make it easy to use the stack. Local variables are automatically located on the stack.

Calling a function automatically adjusts the stack pointer. So does returning.

Stack memory can also be manually allocated with the `alloca` function. This is dangerous and will cause undefined behavior if you alloc too much, but there are other APIs that make it safer (e.g., by falling back to `malloc` if needed).

The size of the allocation does not affect the immediate cost. Allocating and freeing an entire stack frame happens at once. If there are 20 variables on the stack, they can all be allocated and freed simultaneously with one subtract and one add.

Stacks (4)

Using the stack has slightly more overhead than static memory because the pointer needs to be maintained.

In practice, on modern systems, this may not materialize in performance very much. Only if there's a data-dependency on the stack pointer.

However, while stack-based memory management is extremely efficient, it has some downsides...

[What are they?]

Downsides of stacks

Stacks are built to assume that, once memory is allocated, it is held onto it until the function eventually returns.

This means that the stack generally does not have "gaps" in it. You can't release higher stack memory for use by a child function. The memory stays in use.

This is a limitation for any kind of program that needs to hold things, not based on when functions are called and returned, but for as long as necessary.

E.g., how long should a page of text be stored in a word processor? Until the user deletes it, probably. And if they do, are you going to delete all the other pages created after it? Probably not. You want there to be a gap that could be used by other pages or other data.

Questions?

Dynamic allocation

Sometimes we can't use any of the easy strategies:

- Something could use vastly different amounts of memory. Especially creative software, such as word processors, sound and video editors, level editors in games, etc. So static memory is not appropriate.
- The memory could be needed for an unpredictable amount of time, but when it is no longer needed, we don't want to have to free other memory. So the stack is not appropriate.

If both of these circumstances apply, we typically need to use dynamic allocation.

Dynamic allocation (2)

The most basic primitives for dynamic allocation in C are `malloc` and `free`.

Calling `malloc` earmarks some memory as being "used", and gives out a pointer to it.

Calling `free` returns that memory to being unused, so that it can be used by someone else.

Forgetting to call `free` leads to a memory leak, where the memory is not really being used but stays permanently locked up.

Dynamic allocation (3)

There are several drawbacks to dynamic allocation:

1. You must remember to free. It's a common bug to forget. Use-after-free and double-free are less common but also more destructive and hard to find.
2. They are typically slower. There is a data structure that stores free regions of memory.
3. They become fragmented. Suppose I have 1 gigabyte of memory. I allocate a bunch of one 1 megabyte chunks until I've used it up. Then eventually I free two of them. If those two chunks are not next to each other, I cannot allocate a 2 megabyte array, even though I technically have 2 megabytes free.

There are ways to fix all of these, but each fix also comes with drawbacks.

Dynamic allocation (4)

Languages with manual memory management, such as C, C++, and Rust, are often thought of as *fast* languages.

This is obviously a loose statement: you can write slow code in any language, and it's absolutely possible to write code in Rust that is slower than Python.

However, manually managing memory allows the programmer to avoid the overhead of automatic strategies such as garbage collection.

But still, dynamic allocation, even in a language with manual memory management, is often much slower than stack or static memory. Even a call to `malloc` carries significant overhead that still matters to some applications (especially games and drivers).

Dynamic allocation (5)

But first,

[what happens when we call `malloc`? Where does it get the memory that it returns?]

Operating systems

In the old days, operating systems were basically program loaders. Most of the time, your program controlled the whole computer.

In this case the "heap" was just a big area of memory you could use. *

Early multi-tasking systems required more discipline. Operating systems needed to keep track of which system memory was being used by which process. This necessitated an API that a process could use to ask to be given control over a chunk of memory.

*: The fact that it is called "the heap" is confusing. As far as I'm aware, the heap data structure was never used to manage this memory, although I could be wrong.

MMap

On modern POSIX systems, one API for this is `mmap` and `munmap`. Another is `sbrk`.

Modern processors group memory into pages for determining which chunk of memory belongs to which process. A page is typically 4 kilobytes on x86 systems and 64 kilobytes on ARM systems.

Mmap does a lot of things, but one of them is to request the operating system give you one or more pages of memory directly. This causes a page of memory of physical ram to be assigned to a range of virtual addresses in your process.

This completely bypasses `malloc`. It's a way of requesting memory from the OS directly. The Win32 equivalent is `VirtualAlloc`.

The basic operation of an allocator

So in order to handle dynamic allocation, we need some memory that is "ours". In modern systems, we can request that memory with `mmap` or `VirtualAlloc`.

But once we have that memory, how do we manage it? Do we just use a whole page every time someone wants a 16-byte linked-list node?

Of course not. Instead, we maintain a data structure that stores contiguous chunks of memory.

When we need 16-bytes, we need to somehow "carve-out" 16-bytes from a contiguous region. Then when we don't need it anymore, we need to give it back.

Questions?

Early allocators

In the old days, the most basic kind of allocator was the free list.

The "heap" was actually a linked list of regions. Each region had an address, and its size could be determined by looking at the address of the next block.

[In fact, this was how the first K&R C allocator worked](#). It was just circular linked lists of contiguous regions and allocated chunks.

It could start off with one giant free region. Then, when you allocate, it increases the address of the region, which makes it smaller and inserts an allocated region before it.

When you free, it would re-insert a free node back into the list. If two free nodes were next to each other, it would merge them together.

Problems with early allocators

First, there is the overhead of a linked-list node for every allocation and dis-contiguous free region.

Then, Imagine what would happen in an environment with lots of dynamic allocations and frees.

As fragmentation happens, we end up with lots of nodes in the list. So allocating a large, new node, needs to slowly traverse the list until it gets past all the fragments.

This means that we have $O(n)$ worst-case behavior in the allocator, where n is the number of frees. Not great.

Trees

There's no requirement that we use a list, though. Trees have been used too.

For example, we can store regions in a binary search tree, ordered by size.

This means we can find the best fit for an allocation in $O(\lg n)$ time, a big improvement.

And yet, it's still so much slower than stack allocation...

Another old allocator

In the old days, many games and productivity software managed their own memory.

[Here](#) and [here](#) are the source files for Doom's allocator. It uses a linked-list based system, too.

Wouldn't that be slow? It turns out, no, but not because linked lists are fast. Instead, because there is a discipline to using dynamic allocation that prefers to allocate in large chunks.

John Carmack's Zones

You can separate static data from data that is freed after a level or a frame by "tagging" a particular allocation.

There were a handful of pre-defined tags:

```
// ZONE MEMORY
// PU - purge tags.
// Tags < 100 are not overwritten until freed.
#define PU_STATIC          1          // static entire execution time
#define PU_SOUND           2          // static while playing
#define PU_MUSIC           3          // static while playing
#define PU_DAVE            4          // anything else Dave wants static
#define PU_LEVEL           50         // static until level exited
#define PU_LEVSPEC         51         // a special thinker in a level
// Tags >= 100 are purgable whenever needed.
#define PU_PURGELEVEL     100
#define PU_CACHE           101
```

John Carmack's Zones (2)

There was even a tag for [Dave](#). He made the status bar among many other things.

The tag system was useful. It let, for example, every chunk of memory associated with a level get freed whenever the level was finished, without the programmer having to manually free them.

The reason it was called a Zone allocator was because it requested all the memory it needed up-front, using malloc. So there was a giant "zone" of memory that it controlled. Then, it carved it up manually.

This strategy is still useful today...

Questions?

Region-based memory

Here we get to the crux of today's talk. One of the major issues with dynamic memory is the idea that every allocation could be for something completely unrelated.

First: this is cache-inefficient.

If you carve out a region of memory for a particular purpose, you can often use a specialized strategy.

This carved out area is called a *region* or a *zone*.

Arenas

The simplest strategy is the *arena*.

An arena is a region of contiguous memory that is freed all at once.

So imagine storing all of the 200 enemies in a level in one giant array. Once the level is over, you can just free the array all at once.

Another example: particles. Many games have particle systems that track the locations of hundreds of small particles. If each of these were allocated with `malloc` / `free` pairs, the overhead would be severe, and there would be lots of tiny gaps in memory.

Arenas (2)

So keeping these life-span related objects together:

- reduces the cost of `malloc` (because you can use certain strategies when you know that everything will be the same size)
- reduces the cost of `free` because you can free all at once, and because you can often re-use an existing space
- reduces fragmentation, because the arena is a giant chunk of memory.
- Improves cache efficiency. Usually you want all your objects together so you can iterate over them. If they are adjacent in memory, they will likely be in the same cache page. This is called spatial locality and it has a massive impact on performance.

Special strategies

What's that about reducing the cost of malloc if you know everything is the same size?

Consider this:

If every object you want to allocate (assuming no arrays) is the same size, then there will never be fragmentation. Every gap will be large enough for one thing.

But it gets better: consider if the objects are larger than a pointer. Each gap can store a pointer to the next gap... there are some cool things we can do here.

Pools

There is a special memory allocation strategy that is so extremely useful, it is now standard in even most modern `malloc` implementations.

But even though `malloc` supports it, you can often improve on malloc by making it explicit that you want to use it.

It is called the pool. It is a form of region-based memory in which every object in the region has the same size.

Pool basics

Suppose we decide that our levels need to have enemies.

We decide in advance, that 1024 enemies are way more than we will ever need.

Each enemy takes 64-bytes of data.

That means, we can store all the enemies we need in one, giant 64 kilobyte array.

Pool basics (2)

Let's say we have a simple organization like this:

```
size_t bump = 0; // the index of the next enemy  
Enemy enemies[1024];
```

How do we spawn in an enemy?

Bump allocation

The most basic allocation strategy is called *bump allocation*.

Bump allocation is when you simply have a pointer or index to the next free location, and you increment it. It is almost as fast as stack memory.

So we could do something like this:

```
Enemy* spawn_enemy() {  
    Enemy* enemy = (enemies + bump++); // this is the bump allocation  
    return enemy;  
}
```

Running out of space

This is fine, but what happens if we run out of space? We only have room for 1024 enemies.

Ideally, we would support 1024 *simultaneous* enemies. So if the player defeats some of them, there would then open up room for more.

So how do we *free* an enemy?

The free list

One option is to maintain a list of free spaces or gaps in the array.

So instead of sliding the whole array down (which would be $O(n)$), we leave a gap.

Then, when we need to allocate another enemy, we have a choice: bump, or put the new enemy in a gap where an old enemy used to be.

But this raises a question: how do we track the gaps? Do we scan through the list to look for a gap? Do we store the gaps in a list? If we store the gaps in a list, where does the list live? Do we use malloc for that list? Doesn't that defeat the whole purpose of region-based memory?

Interleaving the list

Instead, there's one very cool technique that we can use: store the pointers to the next free node *inside the list of enemies*.

Like this:

```
Enemy* next_free = NULL;
void free_enemy(Enemy* e) {
    // overwrite the enemy data with a pointer to the next free gap
    *(Enemy**)e = next_free;
    // now, the global next free space is where *this* most recent enemy was
    next_free = e;
}
```

Understanding

There's a lot going on here:

- We keep track of the first gap: `next_free`.
- When we free, `next_free` points to the gap we just freed.
- But at the same time, the cell that stored the enemy will now point to the next gap.
- Next free is a pointer to an enemy in the array. But when we store the next free in the free cell, we need to pretend it's an Enemy**.
- That is, we cast the pointer from pointing at a 64-byte enemy to pointing to the first 8-bytes and assuming that they represent a pointer to the next space.

Therefore, as long as the size of an enemy is large enough to store a pointer, we can interleave the list of free gaps in the same memory that is used to store enemies.

Using the gaps

When we want to allocate, we can check if there are any gaps before we bump:

```
Enemy* spawn_enemy() {
    Enemy* result = NULL;
    // are there any gaps?
    if (next_free) { // allocate from the gap
        result = next_free;
        // make next free point to the pointer that was stored in the empty cell
        next_free = *(Enemy**)next_free;
    } else if (n_enemies < 1024) {
        result = (enemies + bump++);
    } else {
        // out of space!
    }
    return enemy;
}
```


Growing gracefully

This is cool, but what if we still want the ability to grow? To have no hard upper bound on the number of enemies?

Well, we could store the enemy list in an array list, but then, when it filled up, it would need to re-allocate a double-sized list and copy everything over (this is how array lists usually work).

But we don't actually need the enemies to be adjacent in memory. It's nice, but if it causes the occasional re-allocation pause, it might not be worth it.

Using ropes

Sometimes people negatively compare linked lists to arrays. E.g., "you should never use linked lists. Arrays are faster!"

This is true: if you have to pick between exactly two choices: linked list or array, arrays are more often the better choice.

But who says we have to pick one or the other?

Enter the rope: a linked list of arrays.

Note: The Erlang programming language actually stores strings as ropes by default. Each string is an array of adjacent characters, but it has a "next" pointer at the end. Therefore, it's one of the rare languages where string appending is a $O(1)$ operation: just set the next pointer!

Using ropes

Instead of having all our enemies in a single array, we can have a linked list of pages:

```
typedef struct enemy_page_t {  
    Enemy enemies[1023];  
    // only store 1023 to avoid wasting a cache page on "next_page"  
    EnemyPage* next_page;  
} EnemyPage;
```

And we can store our pages in a linked list:

```
EnemyPage *first_page, *last_page;
```

Using ropes

Now when we need to allocate:

1. we first check for any gaps.
2. If there are none, we try to bump allocate.
3. If there's no room, we allocate a new page and bump allocate from it:

```
} else { // out of memory
    EnemyPage* page = allocate_new_page();
    last_page->next_page = page;
    last_page = page;
    bump = 0;
    result = page->enemies + bump++;
}
```

The benefits:

There are some big benefits to doing things this way:

1. All our enemies are in giant cache-contiguous chunks, so when we have to e.g., process AI or draw them, we don't get cache misses.
2. The cost of the linked list is minor. We have a tiny break at the end of each long string of enemies to go to the next page. 1 cache miss per 1023 hits is not bad!
3. We free enemies fast: a tiny linked list operation.
4. Alternatively, we gain the option to free every enemy in giant chunks. Every 1023 frees is replaced with 1 big free.

My pool

To show you how to make a pool along these lines, I implemented one myself.

Check out `code/pool.h` and `code/pool.c` for my implementation.

It ends up being about the same size as Carmack's version, but it has different capabilities. If you need different-sized objects, for example, they have to go in different pools. So it doesn't fulfill the same purpose.

Benchmark results

Here is a simple microbenchmark that shows a comparison as to what the average allocation takes with my pool versus `malloc`. It tests 10000 allocations and deallocations of BST tree nodes.

It's about 2.4 times faster, 46 microseconds instead of 113.

```
Bench: pool allocations.....  
Avg time: 45801.336600 nanos, 14476.111854 stdev  
  
Bench: pool allocation and individual frees.....  
Avg time: 46775.285600 nanos, 8989.162862 stdev  
  
Bench: malloc allocations and frees.....  
Avg time: 113185.646800 nanos, 18811.484887 stdev
```

Benchmark warnings

Of course, microbenchmarks often don't reflect real-world conditions.

For example, for tiny allocations, many `malloc`s use a bitmap to track free spaces. Scanning this bitmap can be extremely fast with SIMD. My solution requires rounding up the allocation size to the size of a pointer, which could massively waste memory.

However, there are reasons to believe that my simple benchmarks actually privilege `malloc`, by doing all the allocations and frees at once, allowing malloc to bump-alloc and merge.

Benchmark warnings (2)

But, on the other hand, `malloc` is doing more than my allocator. My allocator requires you to construct a pool in advance and only make the same size allocation with it.

Huge difference: my allocator is not thread safe. It would be much more complicated, and would likely involve assigning each page to a thread.

Don't read too much into this benchmark. It's just a way of showing you that the default allocator *can* be beaten under highly specific circumstances. It doesn't mean the default is bad (it's not).

It just means that the default needs to consider many scenarios that my simple pool does not. Sometimes optimization is about writing code to specific needs with big assumptions .

Behind the scenes

It's kind of surprising how close they are, though, right?

That's because many modern implementations of `malloc` use pools.

The thing is: they have to choose which pool. Whereas we are selecting the pool in advance. This is where a small advantage can come in.

In fact, the popular `jemalloc` allocator has [separate pools for each power of 2 up to a certain size](#). It has an arena for each thread, and then breaks it into chunks of similar sizes.

Once you hit a maximum size, a tree is used. Jemalloc uses a red-black tree. This rarely happens in most applications. Only for large, odd-sized data, like huge blocks of video, audio, or string data.

The drawbacks: changing style

This approach is cool, but requires some compromises.

Using a single allocator is extremely convenient. Using a pool is less so. You need to create the pool, store it somewhere, and explicitly allocate from it.

You have to re-think how you use memory. It becomes a matter of design to layout your memory usage. You end up engaging in an activity that is similar to the RAM maps of old, where you decide ahead of time what objects will be grouped together in pools.

The dangers: double-free detection

Built-in allocators in debug mode can have some important protections as well, that you might want to adopt.

For example, what happens if you free the same memory twice?

This is very bad for us. We end up storing a gap which points to itself. Now following the linked list will result in an infinite loop.

How can we detect this?

Double-free detection

One way is for each page to maintain a bitmap of allocated cells.

So for each 64-byte enemy, there would be a single bit in the bitmap that tells you whether its allocated.

This bit adds a tiny cost to set and clear when we alloc and free, and a very small memory overhead, but both cases can be guarded with an `#ifdef _DEBUG`.

If someone tries to free something with a 0 bit set, you can panic. This can save tremendous debugging pain.

The still-a-problems: shrinking the pool

What if we grow to accommodate a bunch of enemies and then don't need them anymore?

We could delete a page, but only if it's completely empty.

We can't move random data around: it would invalidate pointers that point to it.

But what if we don't use pointers?

Handles instead of pointer

A handle is a variable that acts like a pointer but isn't one.

For example, we could use something like this:

```
// store this in a c file instead of an h file for encapsulation
typedef struct enemy_handle_t {
    Pool* pool;
    size_t index;
} EnemyHandle;
```

This handle is larger than a pointer. However, It has some advantages. Suppose there were functions `Enemy* lock_handle(EnemyHandle)` , `void unlock_handle(EnemyHandle)` . By requiring the user to "lock" the handle to obtain a pointer, we gain the ability to move the enemy around in memory as long as it is not locked.

Defragmenting

This means that when we free an enemy, we can move the last enemy over the gap instead of needing a free list. Now we have *zero* fragmentation.

Also, we can safely free pages we don't need anymore. Once a page is freed, we can even return it to the operating system by unmapping it, assuming we aren't using `malloc` and `free`.

Firefox, which uses Jemalloc, never seems to do this. When I use Firefox on Linux, I always eventually have to close and restart it because it leaks memory. Frankly, I use too many tabs, but it's still an issue.

Use-after-free detection

Additionally, we can detect another common error: use-after-free. This is when you access memory after you've freed it, and is a common reason for segmentation faults.

For each allocated enemy, we can store a "generation" count. This is an integer. Every time we free that cell, we increment the generation count.

We also store the generation in the handle when it is allocated. If a handle ever has a different generation for the same cell, it means a freed handle is being used.

Reference counting

We can use handles for reference counting. If we store, in the cell, how many handles are pointing at it, we can free it when the number hits zero.

This is kind of a hybrid approach between full garbage collection and manual memory management.

It tends to be faster than full garbage collection overall, but prevents you from having cycles (i.e., a circular list won't work because the count will never reach zero and will leak).

Caching

This ability to move things around also works well using the pool as a cache.

There's an *excellent* article on how this enabled porting *Indiana Jones and the Infernal Machine* to the Nintendo 64: [Florian Sauer, Bringing Dr. Jones to the Infernal Machine](#).

The section on caching is later in the article, but all of it is interesting.

The original PC version targetted computers with 128 megabytes of RAM. The Nintendo 64 was only guaranteed to have 4 megabytes (8 with a RAM expansion).

Using a generic pool-style cache, he could guarantee that exactly 4 megabytes of memory would be used, and shifted the sizes of the different pools for performance. His texture cache relied on moving textures around to avoid leaving gaps, and he is able to do that because textures are only accessed by one function once per frame.

Downsides of handles

They add space and an extra indirection. They can be substantially slower than using raw pointers.

On the other hand, they add tons of flexibility and the potential for error detection.

They kind of let you build in a small amount of automatic memory management, but not with the overhead of full garbage collection or requiring a runtime.

There's always a tradeoff, but it's good to know about the options so you can make the right compromise.

For example, what if you made a pool for objects in a game, but made it clear that accesses were only valid during a certain portion of the frame? When you deleted objects, the list of doomed objects would be added to a list, and the pool could automatically defragment during the period in which access was forbidden.

Conclusion

Hopefully this lecture has given you something to think about.

I am not advocating you to use a custom pool for any of this class's assignments.

In fact, my custom pool was made mostly in one day and is not suitable for any real use without substantial profiling against `malloc` and `free`.

On the other hand, it's interesting to think about how adding assumptions can potentially create a big optimization opportunity.

Questions?